Day 1 Python/Pygame/Snake Game Intro and First Lines

9:45-10:30 - Introductions, Quick version of Obstacle Game

"What Is Python?" → Console → VS Code → File Saving (10:30 AM–12:00 PM)

10:30 – 10:45 AM — Introduction to the Console

Goal: Get students comfortable using the keyboard and the terminal.

- Show where the Windows Command Prompt is
    - Open cmd : Press the Windows button on your keyboard > type cmd > hit enter
    - Command Line Prompts

Highlight: "This is the command line. It talks directly to your computer." "The console is like texting your computer commands."

10:45 – 11:00 AM — Python in the Console

Goal: Introduce interactive Python shell and print(). Introduce the idea that coding languages like python can be used to automate systems and relay information in faster ways. But only if we progarm it to do so.

- Open Python from Start Menu or type python in CMD
- Students type: print("Hello, world!")
- Test typing math operations: 5 + 3 or 10 / 2
- Try errors on purpose (missing quotes, missing parenthesis)

Quick Activity:

- Print your name
- Type a sentence with print()
- Write 2 math problems and solve them

Discussion Prompt: "What's different about typing in this window vs regular typing?"

11:00 – 11:30 AM — Transition to VS Code + Running First Program

Goal: Create first script and using VSCode

Working in the command line is awesome but you will notice that once we close the window, everything we've done has disappeared. To give the computer more exciting instructions using the Python language, we need something called a Python script.

A Python script is a list of instructions you write to tell the computer what to do — kind of like writing a recipe or a list of steps for a robot to follow.

Each line in the script is a step, and when you run it, the computer follows those steps one by one. It is a way to make your computer **talk, count, draw, or even play games.**

Let's begin to create our first script. Go ahead and close any open cmd windows.

Open CMD line > Travel to Desktop: cd Desktop > Make a new directory: mkdir Larks Pygame World > Travel into newly created folder: cd Larks Pygame World > Create new File: pygametest1.py > Open VSCode: code .

First look at pygame coding. Create a Window

```
import pygame

pygame.init()
screen = pygame.display.set_mode((480,480))
```

11:30 – 12:00 PM — What just happened?? What is a Game Loop..

What just happened?! The window for our game opened but it immediately disappeared! Well, the program did exactly what we told it to do. No more. No less. If we want the code to work for us longer and more efficiently , we need something called a game loop.

A **game loop** is the part of your game that **runs over and over again**—like a spinning wheel. It tells the computer to **keep checking** what's happening in the game, like:

- Did the player press a key?
- Should the character move?
- Do we need to draw or update anything on the screen?

It runs **many times per second**, so your game feels smooth and alive

https://www.youtube.com/shorts/GsksUwMT5dk

When developing games we need to very clearly outline when to start, what happens when we play and how to stop. Lets create the very basic skeleton for our game loop. Enter the following code:

```
While True:
      For event in pygame.event.get():
            If event.type == pygame.Quit:
                  pygame.quit()
      pygame.display.update()
```

Test our new code > Break for Lunch

<u>Before Lunch Full Code</u>

```
import pygame

# Initialize Pygame
pygame.init()

# Set up the display window
screen = pygame.display.set_mode((480, 480))

# Main loop
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()

    # Update the display
    pygame.display.update()
```

If there is extra time:

In your variables library add background_color = (102,255, 255) > Inside your while True boolean outside of the event loop and before pygame display add screen.fill(background_color)

Recap — You've Covered Before Lunch:

- Console basics (cmd, echo, dir)
- Python interactive shell
- Syntax basics: print(), strings, math
- Common errors
- Intro to VS Code
- Saving .py files
- Running Python files
- Creating Pygame window

- Game Loops

1:00 – 1:45 PM — “If-Else Obstacle Course” Game (Level 1)

Goal: Introduce the concept of control flow using real movement.

- Students act out decisions based on condition cards:
- Reinforce:
    - if condition:
    - else:
- End with group debrief:
  Your script will only do what you command it to do. How did the instructions on the cards limit or expand your power in the obstacle course game? How did it feel?

1:45 – 3:15 PM — Background Set Up: Intro to Variables and Functions

Goal : Create a game window. Begin to understand variables what a Python function is and what it means to call it in a script.

What is a Variable?
A variable in Python is like a labeled box that you can use to store information — numbers, words, or other things you want to use later in your program.
Imagine you have a box labeled "favorite_color" and you put a crayon inside that says "blue".
Later, if someone asks, "What’s your favorite color?", you just open the box and look!
You give the box a name, and you can open it any time to look inside or change what’s in it.

Introduction to Functions

In Python, we use functions to group actions together so we can reuse them anytime we want — like teaching the computer a new word or trick.

What is a Function?
A function is a little block of code that you give a name to. Then, whenever you want the computer to do that action, you just call the function by name.

It's kind of like writing a recipe:

```
def make_pizza():
    print("Roll the dough")
    print("Add sauce")
    print("Add cheese")
```

```
    print("Bake it!")
```

This doesn’t do anything yet. It’s just sitting there, like a recipe card.

Calling the Function

To actually use it, we have to call it by writing its name with parentheses:

```
make_pizza()
```

Now the computer runs everything inside!

Set Up the Game Window

```
import pygame
import sys

# Variables (must come before usage)
WIDTH = 480
HEIGHT = 480
background_color = (135, 206, 235)  # Sky blue

pygame.init()

# Set up the main screen
screen = pygame.display.set_mode((WIDTH, HEIGHT))

# Function to fill the surface with a background color
def drawGrid(surface):
    surface.fill(background_color)

def main():
    pygame.init()

    clock = pygame.time.Clock()
    screen = pygame.display.set_mode((WIDTH, HEIGHT), 0, 32)

    # Create and optimize a drawing surface
    surface = pygame.Surface(screen.get_size())
    surface = surface.convert()

    drawGrid(surface)
    while True:
        clock.tick(10)  # Control the frame rate
        drawGrid(surface)  # Fill the surface with color

        screen.blit(surface, (0, 0))  # Draw the surface onto the
screen
        pygame.display.update()  # Refresh the display

        # Check for quit event
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
main()
```

Students should: Run the file. See the chosen color. Exit properly with user input.

Create Grid For real

Day 1 Full Code

```
import pygame
import sys  # Needed to use sys.exit()

# Start up all the pygame tools
pygame.init()

# COLORS (each color is a mix of red, green, and blue from 0-255)
gray1 = (120, 120, 120)
gray2 = (170, 170, 170)

# WIDTH and HEIGHT of the game window
WIDTH = 480
HEIGHT = 480

# GRID setup
GRID_SIZE = 20  # Each square in the grid is 20x20 pixels
GRID_WIDTH = WIDTH / GRID_SIZE
GRID_HEIGHT = HEIGHT / GRID_SIZE

# This function draws a checkerboard-style grid using two shades of gray
def drawGrid(surface):
    for y in range(int(GRID_HEIGHT)):
        for x in range(int(GRID_WIDTH)):
            # Alternate colors like a chessboard
            if (x + y) % 2 == 0:
                rect = pygame.Rect((x * GRID_SIZE, y * GRID_SIZE), (GRID_SIZE,
GRID_SIZE))
                pygame.draw.rect(surface, gray1, rect)
            else:
                rect = pygame.Rect((x * GRID_SIZE, y * GRID_SIZE), (GRID_SIZE,
GRID_SIZE))
                pygame.draw.rect(surface, gray2, rect)

# MAIN GAME FUNCTION
def main():
    clock = pygame.time.Clock()  # Keeps track of time so the game doesn't run too fast
```

```
    # Create the screen window
    screen = pygame.display.set_mode((WIDTH, HEIGHT), 0, 32)

    # Create a surface the same size as the screen (like a drawing board)
    surface = pygame.Surface(screen.get_size())

    # Make sure the surface is optimized for fast drawing
    surface = surface.convert()

    # Draw the grid onto the surface
    drawGrid(surface)

    # GAME LOOP (runs over and over again)
    while True:
        clock.tick(10)  # Limit to 10 frames per second

        # Redraw the grid every frame
        drawGrid(surface)

        # Paste the surface onto the screen
        screen.blit(surface, (0, 0))

        # Update the screen with everything that's been drawn
        pygame.display.update()

        # Event loop - checks for things like mouse clicks or quit button
        for event in pygame.event.get():
            if event.type == pygame.QUIT:  # If the window close button is clicked
                pygame.quit()
                sys.exit()

# Start the game
main()
```

Parameters

"Alright crew! Who here has ever ordered something at a restaurant?"

(Let kids raise hands or shout out)

"Nice! Now imagine you walk into a restaurant and you say, ‘I want a sandwich!’ 🍞 But the chef stops and says:
‘Okay... what kind of sandwich? With cheese? Toasted? What kind of bread?’

You see, the chef needs *details* to make exactly what you want. Those little details—like the kind of bread or toppings—are what we call **parameters** in Python."

Definition

"A **parameter** is a little piece of information we give to a **function** so it knows *how* to do its job.

Functions are like robots 🦾 or helpers that do things for us. But if we want them to do it *our* way, we have to tell them the details. That’s where parameters come in!"

Indentation

What is indentation?
Indentation just means how far your code is spaced in from the left edge. In Python, we usually use 4 spaces (or one press of the Tab key) to indent a line.

Why does Python care so much about indentation?
Python uses indentation to understand what code belongs together. It helps the computer know:
“Hey, these lines are part of the same team!”

Think of it like a school. You might have different classes: Science, Math, Art. The students in each class sit together in the same group. In Python, indentation is how we tell the computer which students belong to which class.